

▽ 반도체장비예지보전실무_01_연습문제01 개인 의료비용 예측_풀이

```
# 필요한 라이브러리 임포트
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
```

▽ 단계 1: 데이터 로드 및 탐색

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# -----
# 단계 1: 데이터 로드 및 탐색
# -----
# 데이터 파일 'insurance.csv'를 로드합니다.
# 실제 파일 경로에 맞게 수정해야 합니다.
try:
    df = pd.read_csv('/content/drive/MyDrive/_for_exercise/insurance.csv')
except FileNotFoundError:
    print("insurance.csv 파일을 찾을 수 없습니다.")
    # 예시 데이터를 대신 생성하여 실행합니다.
    # data = {
    #     'age': np.random.randint(18, 65, 100),
    #     'sex': np.random.choice(['male', 'female'], 100),
    #     'bmi': np.random.uniform(18, 45, 100),
    #     'children': np.random.randint(0, 5, 100),
    #     'smoker': np.random.choice(['yes', 'no'], 100),
    #     'region': np.random.choice(['northeast', 'northwest', 'southeast', 'southwest'], 100),
    #     'charges': np.random.uniform(1000, 50000, 100)
    # }
    # df = pd.DataFrame(data)

print("===== 데이터셋 상위 5개 행 확인 =====")
print(df.head())
print("\n===== 데이터셋 정보 확인 =====")
df.info()
print("\n===== 수치형 데이터 통계 요약 =====")
df.describe()
```

===== 데이터셋 상위 5개 행 확인 =====

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

===== 데이터셋 정보 확인 =====

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0    age        1338 non-null   int64
1    sex        1338 non-null   object
2    bmi        1338 non-null   float64
3    children   1338 non-null   int64
4    smoker     1338 non-null   object
5    region     1338 non-null   object
6    charges    1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

===== 수치형 데이터 통계 요약 =====

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

▼ 단계 2: 데이터 전처리

```
df_encoded = pd.get_dummies(df, columns=['sex', 'smoker', 'region'],
                             drop_first=True, dtype=int)
df_encoded.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	0	1	0	0	1
1	18	33.770	1	1725.55230	1	0	0	1	0
2	28	33.000	3	4449.46200	1	0	0	1	0
3	33	22.705	0	21984.47061	1	0	1	0	0
4	32	28.880	0	3866.85520	1	0	1	0	0

다음 단계: [df_encoded 변수로 코드 생성](#) [New interactive sheet](#)

```
# 특성(X)과 타겟(y) 분리
X = df_encoded.drop('charges', axis=1)
y = df_encoded['charges']
```

```
print(X.head())
print(y.head())
```

```
print(df_encoded.head())
```

	age	bmi	children	sex_male	smoker_yes	region_northwest \
0	19	27.900	0	0	1	0
1	18	33.770	1	1	0	0
2	28	33.000	3	1	0	0
3	33	22.705	0	1	0	1
4	32	28.880	0	1	0	1

	region_southeast	region_southwest
0	0	1

```

1          1          0
2          1          0
3          0          0
4          0          0
0  16884.92400
1   1725.55230
2   4449.46200
3  21984.47061
4   3866.85520
Name: charges, dtype: float64
   age    bmi  children    charges  sex_male  smoker_yes  region_northwest \
0   19  27.900         0  16884.92400         0           1              0
1   18  33.770         1   1725.55230         1           0              0
2   28  33.000         3   4449.46200         1           0              0
3   33  22.705         0  21984.47061         1           0              1
4   32  28.880         0   3866.85520         1           0              1

   region_southeast  region_southwest
0                0                1
1                1                0
2                1                0
3                0                0
4                0                0

```

```

# 스케일(값의 범위)이 특성마다 너무 다를 때
# -> 정규화(MaxMinScaler) : 값의 범위를 0~1 사이로
# -> 표준화(StandardScaler) : 평균이 0, 표준편차가 1인 분포로 변환

```

```

# 특성 스케일링 (Feature Scaling)
# 선형회귀에서 스케일이 다른 특성들은 가중치(W)에 불균형한 영향을 줄 수 있습니다.
# StandardScaler를 사용하여 평균 0, 표준편차 1이 되도록 조정합니다.

```

```

# 변환기 transformer <- 정규화, 표준화 <- Scaler, 스케일링하려고 변환하는 애들
# fit 함수 // 학습 <- 변환에 쓸 수식을 만드는 거
# transform 함수 // 실제로 변환하는 함수
# fit_transform 함수 // fit+transform이 하는 일을 한꺼번에 해주는 함수

```

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # 결과를 ndarray로 반환

```

```

X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

```

```

# 스케일링된 데이터를 다시 DataFrame으로 변환하여 컬럼명 유지
# X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

```

```

print("\n===== 전처리 후 특성(X) 데이터 상위 5개 행 =====")
print(X_scaled.head())

```

```

===== 전처리 후 특성(X) 데이터 상위 5개 행 =====
   age    bmi  children  sex_male  smoker_yes  region_northwest \
0 -1.438764 -0.453320 -0.908614 -1.010519    1.970587    -0.566418
1 -1.509965  0.509621 -0.078767  0.989591   -0.507463    -0.566418
2 -0.797954  0.383307  1.580926  0.989591   -0.507463    -0.566418
3 -0.441948 -1.385531 -0.908614  0.989591   -0.507463     1.765481
4 -0.513149 -0.292556 -0.908614  0.989591   -0.507463     1.765481

   region_southeast  region_southwest
0        -0.611324         1.765481
1         1.635795        -0.566418
2         1.635795        -0.566418
3        -0.611324        -0.566418
4        -0.611324        -0.566418

```

```

# -----
# 단계 2: 데이터 전처리
# -----
# 범주형 특성(sex, smoker, region)을 One-Hot Encoding으로 변환
# One-Hot Encoding을 사용하는 이유는
# 특정 값에 순서나 서열 관계가 없기 때문입니다.
# 예를 들어, 'northeast'가 'northwest'보다 크거나 작다고 볼 수 없습니다.
# pandas의 get_dummies() 함수를 사용하여 이를 해결합니다.
df_encoded = pd.get_dummies(df, columns=['sex', 'smoker', 'region'],
                             drop_first=True, dtype=int)
# drop_first=True를 설정하여 첫 번째 카테고리(예: sex_female, smoker_no)를
# 제거해 다중공선성 문제를 방지합니다.

# 특성(X)과 타겟(y) 분리
X = df_encoded.drop('charges', axis=1)

```

```

y = df_encoded['charges']

# 특성 스케일링 (Feature Scaling)
# 선형회귀에서 스케일이 다른 특성들은 가중치(W)에 불균형한 영향을 줄 수 있습니다.
# StandardScaler를 사용하여 평균 0, 표준편차 1이 되도록 조정합니다.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # 결과를 ndarray로 반환
# 스케일링된 데이터를 다시 DataFrame으로 변환하여 컬럼명 유지
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("\n===== 전처리 후 특성(X) 데이터 상위 5개 행 =====")
print(X_scaled.head())

```

```

===== 전처리 후 특성(X) 데이터 상위 5개 행 =====
   age      bmi  children  sex_male  smoker_yes  region_northwest \
0 -1.438764 -0.453320 -0.908614 -1.010519    1.970587         -0.566418
1 -1.509965  0.509621 -0.078767  0.989591   -0.507463         -0.566418
2 -0.797954  0.383307  1.580926  0.989591   -0.507463         -0.566418
3 -0.441948 -1.305531 -0.908614  0.989591   -0.507463         1.765481
4 -0.513149 -0.292556 -0.908614  0.989591   -0.507463         1.765481

   region_southeast  region_southwest
0         -0.611324           1.765481
1          1.635795          -0.566418
2          1.635795          -0.566418
3         -0.611324          -0.566418
4         -0.611324          -0.566418

```

▼ 단계 3: 훈련 세트와 테스트 세트 분할

```

# -----
# 단계 3: 훈련 세트와 테스트 세트 분할
# -----
# 데이터를 훈련 세트와 테스트 세트로 나누는 가장 중요한 이유는
# 모델이 학습하지 않은 새로운 데이터에 대해
# 얼마나 잘 작동하는지(일반화 성능) 평가하기 위함입니다.
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
                                                    test_size=0.2, random_state=42)

print("\n===== 훈련/테스트 세트 크기 확인 =====")
print(f'훈련 세트 크기: {X_train.shape}')
print(f'테스트 세트 크기: {X_test.shape}')

```

```

===== 훈련/테스트 세트 크기 확인 =====
훈련 세트 크기: (1070, 8)
테스트 세트 크기: (268, 8)

```

▼ 단계 4: 선형 회귀 모델 훈련

```

# -----
# 단계 4: 선형 회귀 모델 훈련
# -----
# LinearRegression 모델 객체 생성
model = LinearRegression()

# fit() 메서드는 경사 하강법과 같은 최적화 알고리즘을 사용하여
# 예측 오류(손실 함수)를 최소화하는 최적의 가중치(W)와 편향(b)을 찾습니다.
model.fit(X_train, y_train)

print(X.columns)
print(model.coef_)

# 학습된 가중치(W)와 편향(b) 확인
print("\n===== 학습된 가중치(W)와 편향(b) 확인 =====")
print("가중치 (Coefficients):")
for col, coef in zip(X.columns, model.coef_):
    print(f' - {col}: {coef:.4f}')

print(f'\n편향 (Intercept): {model.intercept_:.4f}')
# 각 가중치(model.coef_)는 해당 특성이 타겟에 미치는 영향의 크기와 방향을 의미
# 가중치의 절댓값이 클수록 타겟에 더 큰 영향을 미칩니다.

```

```

Index(['age', 'bmi', 'children', 'sex_male', 'smoker_yes', 'region_northwest',
      'region_southeast', 'region_southwest'],
      dtype='object')
[ 3.60914902e+03  2.05488506e+03  5.12478869e+02 -9.29533694e+00
  9.54425109e+03 -1.58959435e+02 -2.92759080e+02 -3.47270359e+02]

===== 학습된 가중치(w)와 편향(b) 확인 =====
가중치 (Coefficients):
- age: 3609.1490
- bmi: 2054.8851
- children: 512.4789
- sex_male: -9.2953
- smoker_yes: 9544.2511
- region_northwest: -158.9594
- region_southeast: -292.7591
- region_southwest: -347.2704

편향 (Intercept): 13314.3359

```

단계 5: 모델 예측 및 평가

```

# -----
# 단계 5: 모델 예측 및 평가
# -----
# 테스트 세트로 예측 수행
y_pred = model.predict(X_test)
print(y_pred[:5])
print(y_test[:5])

# 모델 평가
mse_test = mean_squared_error(y_test, y_pred)
r2_test = r2_score(y_test, y_pred)

print("\n===== 모델 평가 (테스트 세트) =====")
print(f'테스트 MSE (Mean Squared Error): {mse_test:.2f}')
print(f'테스트 R2 Score: {r2_test:.4f}')

# MSE는 예측값과 실제값의 차이(오차)를 제공한 것의 평균으로,
# 작을수록 좋은 성능을 의미합니다.
# R2 Score는 모델이 타겟 변동성을 얼마나 잘 설명하는지를 나타내며,
# 1에 가까울수록 좋은 성능입니다.

# 훈련 세트에 대한 예측 및 평가
y_train_pred = model.predict(X_train)
mse_train = mean_squared_error(y_train, y_train_pred)
r2_train = r2_score(y_train, y_train_pred)

print("\n===== 모델 평가 (훈련 세트) =====")
print(f'훈련 MSE (Mean Squared Error): {mse_train:.2f}')
print(f'훈련 R2 Score: {r2_train:.4f}')

# 만약 훈련 세트의 R2 Score는 매우 높고 테스트 세트의 R2 Score가 매우 낮다면,
# 모델이 훈련 데이터에 과적합(Overfitting)되었을 가능성이 있습니다.

[ 8969.55027444  7068.74744287 36858.41091155  9454.67850053
 26973.17345656]
764      9095.06825
887      5272.17580
890      29330.98315
1293     9301.89355
259      33750.29180
Name: charges, dtype: float64

===== 모델 평가 (테스트 세트) =====
테스트 MSE (Mean Squared Error): 33596915.85
테스트 R2 Score: 0.7836

===== 모델 평가 (훈련 세트) =====
훈련 MSE (Mean Squared Error): 37277681.70
훈련 R2 Score: 0.7417

```